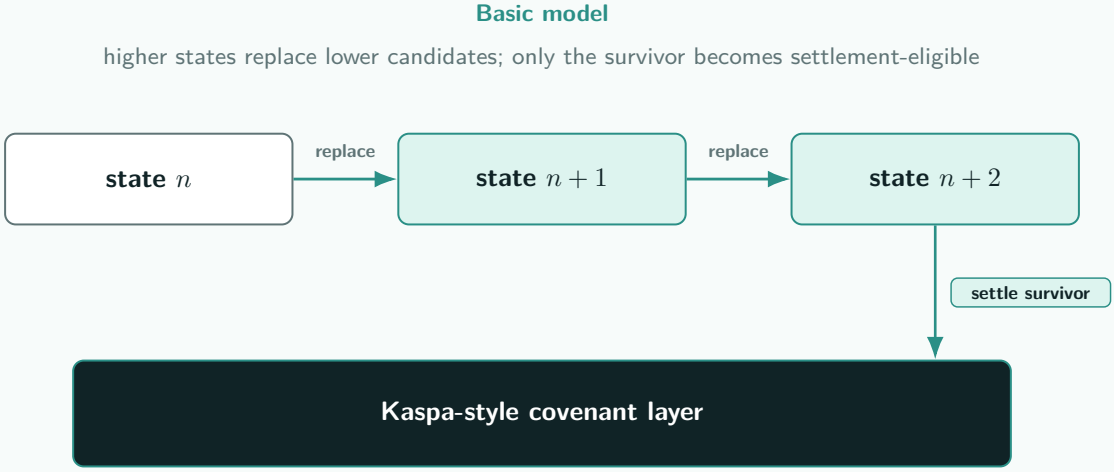
Kurrent

Latest-State Settlement over Kaspa-Style Covenants

A short research note on non-confiscatory off-chain state, covenant-bound transition rules, and response-window settlement on a fast UTXO ledger.

Arthur Zhang



Research note

Contains a normative bilateral-channel specification. Prepared for technical discussion in the Kaspa ecosystem. Not a mainnet-readiness claim.

KIP reference snapshot: `kaspanet/kips@1aba3b8` · Prepared 2026-06-20.

Any KIP text referenced in this note is the version vendored at `kaspanet/kips@1aba3b8`; later KIP revisions may differ.

Abstract

I specify a fixed-participant bilateral channel with a fixed aggregate verification key P_{agg}, secured by a contest-output state machine over Kaspa's DAA-relative sequence maturity: a relative-sequence lock measured in DAA score. The construction is exposed to script through KIP-17 sequence introspection and the KIP-20 covenant-lineage surface. Each state update moves the channel value from a contest output of value V at state n to a successor contest output at state $m > n$, or to a settlement transaction after maturity.

Three primitives carry the normative bilateral channel: a fixed MuSig2 aggregate over the two participant public keys, a versioned scope identifier binding the channel's economic parameters, and a DAA-relative input sequence lock. Cooperative close is an orthogonal fast path requiring no response window; it is disjoint from the unilateral-replacement safety argument.

The safety claim is deliberately narrow. A higher authorised state has an immediately eligible covenant-preserving replacement path, while settlement of the current contest output is relatively delayed; stale-state safety is conditional on the higher replacement becoming accepted before stale settlement is accepted. The construction is non-confiscatory rather than watch-free: it removes revocation-secret forfeiture, but still depends on active monitoring and timely inclusion inside the response window. A prototype harness exercising the marker-and-verifier path is reported as evidence of model-level displacement behaviour; it does not implement the normative contest-output transaction graph.

1 Scope

This note specifies the normative protocol construction for a bilateral latest-state settlement channel on Kaspa, post Crescendo 10 BPS [5]. The construction is:

- Fixed-participant (two parties: A and B);
- Fixed aggregate verification key P_{agg} (32-byte x-only BIP-340, deterministic MuSig2 over lex-sorted participant public keys);
- Single aggregate Schnorr signature per state certificate;
- Single normative state machine: OpenOutput $\to$ ContestOutput(n) for any valid n with $0 \le n \le 2^{63} - 1$, with ContestOutput(n) $\to$ {ContestOutput(m), Settlement(n)};
- Two cooperative-close fast paths: OpenOutput $\to$ CoopClose and ContestOutput(n) $\to$ CoopClose(n).

Out of normative scope (relocated to dedicated sections below): marker / KIP-21 observability, factories, sum commitments, hashlock interop, zk proof systems. The prototype harness exercises the marker path; the normative specification does not depend on it. Factory material is research-only in this note: it records materialisation invariants and proof obligations, not a production compressed-commitment construction.

Reader map. This note uses *contest output* for the singleton channel UTXO currently under dispute. A *state certificate* is the 4-tuple $(\mathsf{scope\_id}, n, \mathsf{state\_root}_n, \sigma_n)$ under the channel's fixed MuSig2 aggregate verification key, where σ_n is an aggregate Schnorr signature over the certificate message $m_n = H_{\text{KurrentStateCert/v1}}(\mathsf{scope\_id} \,\|\, \text{le64}(n) \,\|\, \mathsf{state\_root}_n)$. DAA-relative maturity is relative-sequence maturity measured against the contest output's creation DAA score; the settlement branch is invalid until that score plus Δ. The covenant ID is the KIP-20 lineage label carried by the channel UTXO. The marker / KIP-21 path is prototype evidence and observability infrastructure, not the bilateral fund-safety core.

2 The Problem: Keeping an Old State Dead

Off-chain protocols are built on a bargain. Most activity should remain local to the parties who care about it, whilst the public ledger should be invoked only for settlement, dispute, or anchoring. This bargain is attractive because it preserves the scarce resource that a base layer provides: globally agreed ordering and validation. It is also dangerous, because a locally negotiated state is not useful unless every participant can later force the correct outcome on-chain.

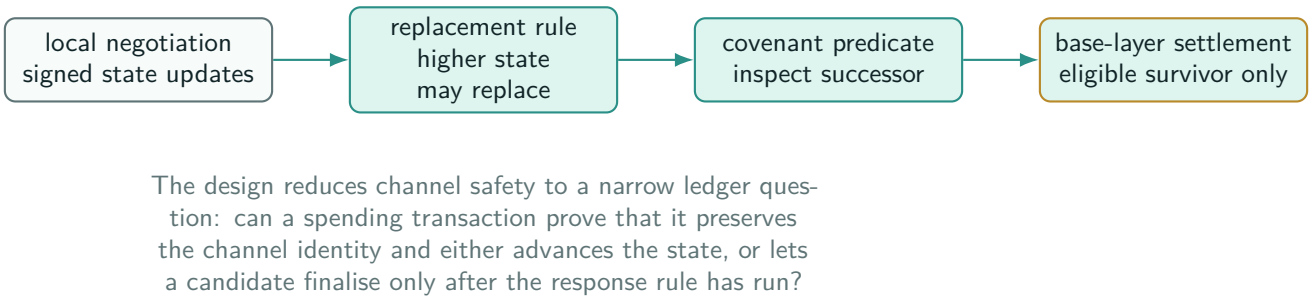


Figure 1: *The latest-state thesis: off-chain agreement is cheap; on-chain enforcement is narrow and rule-bound.*

The hard question is therefore not how to update a balance sheet. It is how to stop an older balance sheet from coming back to life. In a channel, Alice and Bob may agree successively to states $S_0, S_1, S_2, \ldots, S_n$. State S_i may have been valid when it was signed. After S_{i+1} is accepted, however, S_i is no longer the contract. The ledger must not be fooled by the mere history of S_i having been signed.

The eltoo idea is narrow: a later authorised state should supersede an older settlement candidate by state number rather than by confiscatory revocation [1]. Kurrent maps that discipline onto a contest-output state machine over one relative-sequence lock and one aggregate signature. A higher valid state displaces a lower one only if the higher spend consumes the contest UTXO first; after maturity, consensus simply decides which conflicting spend it accepts. The design therefore changes the stale-state consequence, but does not remove the liveness race.

3 From NOINPUT to Covenant-Bound Replacement

Bitcoin eltoo realises that discipline through Bitcoin-specific transaction rebinding: NOINPUT-style floating settlement lets a later update bind to a state lineage without committing to one exact previous transaction identifier [1]. Perun and the virtual-state-channel line form a parallel research direction, studying multi-party virtual-channel settings in which off-chain state objects are composed and settled through formally specified channel protocols [12]. Kurrent's mechanism differs from eltoo in that it does not need a Kaspa equivalent of Bitcoin's floating transaction primitive. It expresses replacement through KIP-20 covenant lineage, KIP-17 introspection, successor predicates, participant authorisation, and DAA-relative maturity. Covenant identity is an output-level lineage anchor: it identifies the continuing object, while the successor and maturity predicates decide which spends are eligible.

A Kurrent state is bound by a strict progress relation against the spent contest output. The state number n lives in a state certificate

$$\mathsf{StateCert}_n = (\mathsf{scope\_id}, n, \mathsf{state\_root}_n, \sigma_n),$$

where $m_n = H_{\mathrm{KurrentStateCert/v1}}(\mathsf{scope\_id} \parallel \mathrm{le64}(n) \parallel \mathsf{state\_root}_n)$ is the certificate message and σ_n is an aggregate Schnorr signature produced by the MuSig2 protocol over m_n by the fixed participant set. Participant public keys are lexicographically ordered before deterministic key aggregation. A presented successor must satisfy the same $\mathsf{scope\_id}$; a strictly higher n with $n \leq 2^{63} - 1$; a valid certificate; principal conservation $v_A + v_B = V$; and outputs whose slots hash to the bound state root under §5.2. The chain-extension path and the displacement path share these predicates. A displacement spend, as a higher-numbered state replacing a lower candidate, is predecessor-independent: it need not be the immediate successor of the spent state.

The design question is therefore precise: can the Kaspa covenant surface described by the KIP-17/KIP-20 direction express the required eltoo-like safety properties without a floating transaction primitive? A credible answer must show how the state-bearing output exposes its identity, how the spending transaction proves strict progress, how conservation is checked, and how the ledger environment decides whether a higher valid candidate displaced a lower one in time. Sections 5–7 state the answer and discharge the sufficiency claim rather than assuming it.

4 Why Kaspa-Style Covenants

The UTXO model is deliberately austere. A coin is an output guarded by a spending condition. A transaction consumes old outputs and creates new outputs. This simplicity is one reason for the robustness of UTXO systems, but it also makes stateful protocols awkward: the state is not stored in an account; it is carried by the lineage of outputs.

Kaspa's scripting direction changes the design space. KIP-10 introduces transaction introspection opcodes, allowing a script to inspect the spending transaction's input and output counts, current input index, input and output values, and script public keys [4]. KIP-17 extends that surface to full covenant support by adding introspection over further transaction fields, byte-string manipulation primitives, hash opcodes, and larger post-Toccata script-element limits [7]. KIP-20 then gives covenants a first-class lineage primitive: a consensus-tracked covenant identifier carried by UTXOs and declared by outputs, so that a covenant instance has an identity that is not merely inferred from recursive witness data [8].

At the pinned KIP snapshot `kaspanet/kips@1aba3b8`, three consensus primitives carry the entire Kurrent construction: KIP-17 script introspection (sequence fields, output count, output values, output script public keys); KIP-20 covenant context (covenant identifier, authorised-output count and index, covenant-wide input and output counts); and Kaspa's DAA-relative sequence maturity, exposed through KIP-17 sequence introspection. No marker, no lane, no global ordering proof is required for bilateral fund safety. The post-Toccata partitioned sequencing surface in KIP-21 [9] is valuable for observability, watchtower evidence, and future based-app and compressed-factory paths, but it is not a fund-safety primitive for the bilateral channel. Even though KIP-21 is present in the pinned snapshot, it remains an observability and compressed-evidence surface here, not a bilateral fund-safety requirement. A latest-state channel needs identity, inspection, transition, and the relative-sequence maturity rule; it does not need a global ordering commitment as a fund-safety gate. The broader Toccata narrative frames covenants and zk-oriented scripting as part of Kaspa's application-development trajectory [10, 11].

5 Commitment and Authorisation Model

The protocol-level commitment hierarchy splits the wire fields into three layers, each bound by a single canonical hash. The lowest-level fields are folded into a static *scope identifier*; the state-dependent economic rights are folded into a *state root*; the participant authorisation binds only the scope, the state number, and the state root. The result is a constant-size signed payload under a fixed participant set and a canonical commitment structure; the latter admits direct displacement without an ancestry obligation.

5.1 Scope Identifier

The channel is identified by a 32-byte scope identifier:

$$\mathsf{scope\_id} \;=\; H_{\mathrm{KurrentScope/v1}}(\mathsf{ScopeEncoding})$$

with

$$\begin{aligned}\mathsf{ScopeEncoding} \;=\; &\mathsf{chain\_context}_{32} \,\|\, \mathsf{covenant\_id}_{32} \\ &\|\, \mathsf{agg\_key}_{32} \,\|\, \mathrm{le32}(\Delta) \\ &\|\, \mathrm{le16}(\mathsf{programme\_version}) \,\|\, \mathsf{policy\_hash}_{32}\end{aligned}$$

$\mathsf{chain\_context}_{32}$ binds the scope to a specific network identity (e.g., a hash of the genesis parameters), preventing a scope constructed on one chain from being replayed against the same outpoint on another chain. $\mathsf{agg\_key}_{32}$ is the 32-byte x-only BIP-340 aggregate verification key produced by deterministic MuSig2 [3] key aggregation over lexicographically ordered participant public keys. $\mathsf{policy\_hash}_{32}$ is the v1 commitment to a canonical binary policy structure (§5.5). Two-stage initialisation: covenant_id is derived first via KIP-20 from the authorising outpoint and initial authorised outputs; scope_id is then derived using that covenant_id. If the transaction carrying the authorising outpoint disappears from the accepted view during a reorganisation before it reaches the deployment's finality policy, the derived covenant_id and scope_id are abandoned with that view. A scope is live only if it is carried by an accepted and finalised covenant lineage in the current view; replaying a pre-reorg scope would require the same chain context, the same outpoint-derived covenant id, the same authorised outputs, the same aggregate key, and the same policy hash in a validation view where that lineage exists. Domain tags are ASCII strings used directly as BLAKE2b keyed-mode keys via the script opcode

$$\mathrm{OpBlake2bWithKey}(\mathrm{payload},\ \mathrm{domain\_tag}),$$

with key length up to 64 bytes. The active hash domain tags are:

`KurrentScope/v1`, `KurrentState/v1`, `KurrentStateCert/v1`,
`KurrentCoopCloseOutputs/v1`, `KurrentCoopClose/v1`.

A sixth tag `KurrentPolicy/v1` is reserved for the policy encoding (§5.5). The BLAKE2b-256 keyed mode is the standard RFC 7693 §2.5 keyed construction; BLAKE3 does not appear in the Kurrent core.

Canonical contest program encoding. The phrase "canonical contest redeem-script preimage" means the deterministic byte encoding

$$\mathsf{ContestScript}_n = \mathsf{ContestScript}(\mathsf{programme\_version}, \mathsf{chain\_context}_{32}, P_{\mathrm{agg}}, \mathsf{policy\_hash}, \Delta, n, \mathsf{state\_root}_n, \mathsf{settlement\_mask}_n)$$

of the v1 contest program. It includes the fixed branch structure for replacement, settlement, and cooperative close; the fixed aggregate key P_{agg}; the committed Δ; the authenticated state number n; the authenticated $\mathsf{state\_root}_n$; the authenticated settlement-presence mask; the domain tags above; and the exact opcode-level encodings used by the predicate. It intentionally does not embed $\mathsf{scope\_id}$, because $\mathsf{scope\_id}$ contains the KIP-20 covenant id and the genesis id itself is derived from the initial authorised-output set. Instead, every spend recomputes $\mathsf{scope\_id}$ from the input's actual introspected covenant id, the authenticated static constants, and the policy hash. The spent input's P2SH commitment authenticates the revealed $\mathsf{ContestScript}_n$; therefore n, $\mathsf{state\_root}_n$, and $\mathsf{settlement\_mask}_n$ are script constants, not free witness fields. A successor ContestOutput(m) must pay to P2SH($\mathsf{ContestScript}_m$), where $\mathsf{ContestScript}_m$ contains authenticated m, $\mathsf{state\_root}_m$, and $\mathsf{settlement\_mask}_m$. Output 0's script public key is compared against that exact P2SH encoding.

5.2 State Root

Each state n binds to a 32-byte state root:

$$\mathsf{state\_root}_n \;=\; H_{\mathrm{KurrentState/v1}}\big(\mathsf{settlement\_mask}_n \,\|\, \mathsf{0x00} \,\|\, \mathrm{commitSPK}(\mathrm{spk}_A) \,\|\, \mathrm{le64}(v_A) \,\|\, \mathsf{0x01} \,\|\, \mathrm{commitSPK}(\mathrm{spk}_B) \,\|\, \mathrm{le64}(v_B)\big)$$

where

$$\mathrm{commitSPK}(\mathrm{spk}) \;=\; \mathrm{le16}(\mathrm{version}) \,\|\, \mathrm{le64}(\mathrm{len}(\mathrm{script})) \,\|\, \mathrm{script}.$$

Equivalently, the state-root preimage layout is:

```
mask || 0x00 || commitSPK(spk_A) || le64(v_A)
   || 0x01 || commitSPK(spk_B) || le64(v_B).
```

The fixed-width commitSPK form is the canonical commitment form used inside every signed state-root slot and inside the cooperative-close participant-output hash. It is not the byte vector returned by Toccata output introspection. Whenever the covenant compares an actual transaction output, it uses Toccata's script-public-key byte form

$$\mathrm{toccataSPK}(\mathrm{spk}) \;=\; \mathrm{be16}(\mathrm{version}) \,\|\, \mathrm{script},$$

which is exactly the value pushed by OpTxOutputSpk. For a redeem script R,

$$\mathrm{P2SH}(R) = \mathrm{toccataSPK}\big(\mathrm{ScriptPublicKey}(0,\ \mathrm{OpBlake2b} \,\|\, \mathrm{OpData32} \,\|\, \mathrm{BLAKE2b256}(R) \,\|\, \mathrm{OpEqual})\big).$$

The conservation invariant $v_A + v_B = V$ is enforced by the covenant. Because Kaspa rejects zero-valued transaction outputs, settlement uses a canonical presence mask:

$$\mathsf{settlement\_mask}_n = \begin{cases} \mathsf{0x01} & v_A = 0,\ v_B = V, \\ \mathsf{0x02} & v_A = V,\ v_B = 0, \\ \mathsf{0x03} & v_A > 0,\ v_B > 0, \\ \mathsf{0x00} & \text{invalid.} \end{cases}$$

The mask is authenticated by both the state root and the contest script. For $\mathsf{0x03}$, settlement outputs are A at index 0 and B at index 1; for $\mathsf{0x02}$, the sole participant output is A at index 0; for $\mathsf{0x01}$, the sole participant output is B at index 0. Optional sponsor change follows the participant outputs. Replacement and opening transactions place the successor contest output at output index 0 and optional sponsor change at output index 1. No address sort is used; participant slots are positional, not lexical.

5.3 State Certificate and Aggregate Signature

For each state n:

$$m_n \;=\; H_{\mathrm{KurrentStateCert/v1}}(\mathsf{scope\_id} \,\|\, \mathrm{le64}(n) \,\|\, \mathsf{state\_root}_n)$$

$$\sigma_n \;\leftarrow\; \mathrm{MuSig2Sign}\big((sk_A, sk_B),\ m_n\big)$$

On-chain validity: $\mathrm{SchnorrVerify}(P_{\mathrm{agg}}, m_n, \sigma_n) = 1$. In the Toccata opcode profile, this abstract check is realised by OpCheckSigFromStack over a 32-byte message hash; the raw stack values consumed are signature, message hash, and public key. The state number is constrained to $0 \le n \le 2^{63} - 1$ so that OpBin2Num and the script arithmetic opcodes interpret it as a signed i64 without overflow; the bound is set by the script-number encoding and the full range is operationally usable, with $2^{63} - 1$ itself a valid state number rather than a reserved sentinel. σ_n is an aggregate Schnorr signature produced by MuSig2 [3]. Participant public keys are lexicographically ordered before deterministic key aggregation, and rogue-key resistance is provided by the MuSig2 key-aggregation procedure. The signing nonce and commitment rounds remain off-chain; only the resulting aggregate σ_n is bound on-chain. This is a liveness limitation, not a consensus ambiguity: if a participant is offline or refuses the nonce/signing round, no new state certificate is produced; the latest already-signed certificate remains the current state for the unilateral contest and settlement paths, which operate against the same contest UTXO without further authorisation from the offline party. A non-interactive emergency key, threshold fallback, or script-visible multisig alternative would be a different authorisation policy and must be committed through the policy hash; it is not an implicit runtime fallback of the fixed MuSig2 mainline.

Same-number equivocation is forbidden by signer policy, not resolved by the covenant after the fact. Each honest signer durably records the highest state number it has signed for a given scope_id and signs at most one state_root for that number. It signs a new certificate only for n strictly greater than the recorded value. Two different roots at the same $(\mathsf{scope\_id}, n)$ cannot replace each other because neither satisfies strict progress.

5.4 Cooperative-Close Message

For a cooperative close over a specific input outpoint I and settlement-output hash S:

$$S = H_{\mathrm{KurrentCoopCloseOutputs/v1}}\big(\mathsf{settlement\_mask} \,\|\, \mathrm{commitSPK}(\mathrm{spk}_A) \,\|\, \mathrm{le64}(v_A) \,\|\, \mathrm{commitSPK}(\mathrm{spk}_B) \,\|\, \mathrm{le64}(v_B)\big).$$

The authorisation message is:

$$m_{\mathrm{coop}} = H_{\mathrm{KurrentCoopClose/v1}}(\mathsf{scope\_id} \,\|\, I \,\|\, S)$$

$$\sigma_{\mathrm{coop}} \leftarrow \mathrm{MuSig2Sign}\big((sk_A, sk_B),\, m_{\mathrm{coop}}\big)$$

The signature binds to the exact input outpoint; it cannot be replayed against a later contest output in the same channel lineage. Cooperative close is an orthogonal fast path (§6) and is disjoint from the unilateral-replacement safety argument of §7.

5.5 Settlement Policy

Although policy_hash binds into scope_id and therefore into the channel authorisation, the policy fields are not repeated inside $\mathsf{StateCert}_n$. They are deployment parameters that define the scope itself: a deployment that changes coop_close_enabled or selects a different settlement_shape_id produces a different scope_id and is treated as a distinct channel scope.

$$\begin{aligned}\mathsf{PolicyEncoding} = {} & \mathrm{le16}(\mathsf{policy\_version}) \,\|\, \mathrm{le32}(\mathsf{sponsor\_policy\_id}) \\ & \|\, \mathrm{le32}(\mathsf{settlement\_shape\_id}) \,\|\, \mathsf{coop\_close\_enabled}_1 \,\|\, \mathsf{reserved}_{64}\end{aligned}$$

with v1 assignments: $\mathsf{policy\_version} = 1$, $\mathsf{sponsor\_policy\_id} = 0$ (external-sponsor-only for positive-fee protocol transactions), $\mathsf{settlement\_shape\_id} = 1$ (two-party mask-bearing shape), $\mathsf{coop\_close\_enabled} \in \{0, 1\}$, $\mathsf{reserved}_{64}$ must be all-zero (rejection on non-zero is normative).

$$\mathsf{policy\_hash} = H_{\mathrm{KurrentPolicy/v1}}(\mathsf{PolicyEncoding})$$

When a branch needs an individual policy field, the witness supplies the canonical PolicyEncoding and the script recomputes policy_hash before reading that field; alternatively, a production script profile may embed the fixed policy fields as constants and compare the resulting hash to the committed policy_hash. In either case, a branch never accepts an unbound policy field. programme_version identifies the normative protocol ABI: state encoding, certificate message, branch predicates, output shape, sequence encodings, policy encoding, numeric bounds, and verification semantics. It is bumped only when the set of valid protocol spends or the canonical bytes being signed/hashed changes. Implementation-only refactors, test harness changes, or equivalent script optimisations that preserve the protocol ABI do not bump programme_version; they are tracked by source commit, build profile,

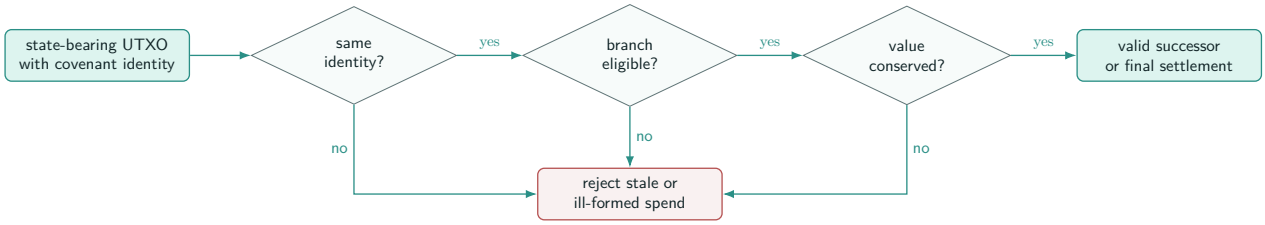


Figure 2: *The covenant predicate is a public, deterministic, narrow rule. Invalid spends are rejected because they fail the relevant identity, branch-eligibility, or conservation check; valid spends are replacement, settlement, or cooperative close.*

script hash, or deployment artefact hash.

6 Normative Transaction State Machine

The state machine has three branches off the long-lived contest output (unilateral replacement, unilateral settlement, cooperative close), plus an opening transition from the initial OpenOutput that produces the first ContestOutput, and a fast cooperative-close path that can also spend the OpenOutput without a response window. Cooperative close is an orthogonal fast path that is disjoint from the unilateral-replacement safety argument. Bounded transaction shapes are stated for each branch.

Figure 2 is schematic. For replacement, branch eligibility is strict state progress ($m > n$); for settlement, it is relative-sequence maturity; for cooperative close, it is joint authorisation bound to the exact input outpoint.

All normative protocol transactions use tx.version = TX_VERSION_TOCCATA = 1, tx.lock_time = 0, channel input index 0, and deterministic encodings for outpoints, sequence fields, values, and script-public-key commitments. Optional sponsor inputs, when present, are at input index 1; optional sponsor change is the last output. If no sponsor is present, the transaction input and output counts are reduced accordingly.

6.1 Funding Genesis

The KIP-20 genesis transaction is the base case for the singleton-lineage argument. It consumes one or more ordinary funding inputs and creates exactly one Kurrent covenant output:

$$\text{FundingInputSet} \longrightarrow \text{OpenOutput}(V).$$

The ordered initial authorised-output set used for KIP-20 covenant-id derivation contains exactly output 0. By convention, the funding input that authorises the Kurrent covenant output is placed at input index 0; any additional funding, sponsor, or change-management inputs are placed at indices ≥ 1. Output 0 has value V, declares covenant binding ($\mathsf{covenant\_id} = id, \mathsf{authorising\_input} = 0$), and pays to the canonical open script for the agreed static parameters. That open script embeds $\mathsf{chain\_context}_{32}$, P_{agg}, Δ, $\mathsf{programme\_version}$, and $\mathsf{policy\_hash}$, but not $\mathsf{scope\_id}$; when it is spent, it recomputes $\mathsf{scope\_id}$ from OpInputCovenantId(0) and those authenticated constants. The funding ceremony requires both participants to obtain and persist at least one valid state certificate for the intended opening state before the funding transaction is broadcast.

The funding construction rules are: the transaction has one or two outputs, output 0 is the sole Kurrent covenant output, output 0 carries covenant binding $(id, 0)$, and output 0 pays to P2SH of the canonical open script. In Toccata, genesis covenant bindings are validated by recomputing the KIP-20 covenant id from the authorising input outpoint and the ordered authorised-output set; genesis relations are not inserted into the continuation-only OpCov$*$ or OpAuth$*$ script contexts. The optional second output is sponsor or ordinary change and has no Kurrent covenant binding. Genesis creates the single live Kurrent covenant UTXO; every continuation branch below preserves or terminates that singleton.

6.2 Open Output

The initial covenant output of value V at fixed script is not spendable unilaterally; it is spendable only by:

- CoopClose (fast path), or
- ContestOpen(n) for any valid n with $0 \leq n \leq 2^{63} - 1$.

6.3 Contest Opening

Spends OpenOutput. The single covenant output is ContestOutput(n) for the state number n authorised by the opening certificate, with value V and the same covenant ID. Covenant predicates:

- OpCovInputCount(id) = 1,
- OpCovOutputCount(id) = 1,
- OpCovInputIdx($id, 0$) = i where i is the channel-input index, so the open input is the unique covenant-authorising input for this lineage,
- OpCovOutputIdx($id, 0$) = 0, so the sole covenant output is at output index 0,
- OpAuthOutputCount(i) = 1 for the channel input,
- OpAuthOutputIdx($i, 0$) = 0, so the channel input's sole authorised output is at output index 0,
- OpOutputCovenantId(0) = id and OpOutputAuthorizingInput(0) = i, so output 0 directly declares continuation under the same covenant id and this input as its authoriser,
- successor output 0 has value V and OpTxOutputSpk(0) = P2SH($\mathsf{ContestScript}_n$); the witness exposes the opening certificate's settlement slots (spk_A, v_A) and (spk_B, v_B), the covenant recomputes $\mathsf{state\_root}_n$ and $\mathsf{settlement\_mask}_n$ from those slots, rejects mask 0x00, and checks $v_A + v_B = V$,
- opening certificate ($\mathsf{scope\_id}, n, \mathsf{state\_root}_n, \sigma_n$) validates, with m_n as defined in §5.3:

$$\text{SchnorrVerify}(P_{\text{agg}},\ m_n,\ \sigma_n)\ =\ 1,$$

- $0 \le n \le 2^{63} - 1$ (so that the script arithmetic opcodes interpret n as a signed i64 without overflow),
- $\text{SEQ}_{\text{open}} = \text{le64}(2^{63})$ (disable bit set, every other bit zero).

Bounded shape:

- OpTxInputCount $\in \{1, 2\}$ and OpTxOutputCount $\in \{1, 2\}$.
- Inputs: [open input at idx 0, optional sponsor input at idx 1].
- Outputs: [contest output of V at idx 0, optional sponsor change at idx 1].
- Sequence: open input and sponsor input both use le64(2^{63}) (disabled relative sequence); tx.lock_time = 0.
- Sponsor invariant: sponsor_input = sponsor_change + fee.

The opening branch is parameterised by n, not fixed to $n = 0$: the participants may open the channel directly at the latest state they have agreed on, including the first joint signature, without first committing to a fresh 0-state. This avoids using a dedicated 0-state merely to enter the contest graph, at the cost of making watchers treat the opening certificate as the first public state rather than assuming that every channel begins at 0. Under the fixed-principal profile, the sponsor input is mandatory for any positive-fee opening by algebraic consequence of the conservation rule $v_A + v_B = V$.

6.4 Replacement (predecessor-independent direct displacement)

Spends ContestOutput(n). Produces ContestOutput(m) with $m > n$, $m \le 2^{63} - 1$. The replacement branch is the normative path for arbitrary higher-state displacement before stale settlement is accepted: a higher-numbered valid state $m > n$ can displace a lower candidate n without binding the chain of intermediate states, because the contest UTXO is the single UTXO being spent. The current local-devnet evidence path of the prototype marker harness covers adjacent-state displacement only, where each published higher candidate advances the state chain by exactly one and binds the previous commitment. The normative contest-output path does not require ancestry evidence. An ancestry-proof or intervening-chain-disclosure requirement applies only to the marker-and-verifier evidence path, where the harness records candidate history rather than consuming the contest UTXO directly; that evidence-path limitation is recorded in `KURRENT_SECURITY_ASSUMPTIONS.md`. Covenant predicates:

- OpCovInputCount(id) = 1,
- OpCovOutputCount(id) = 1,
- OpCovInputIdx($id, 0$) = i where i is the channel-input index, so the channel input is the unique covenant-authorising input for this lineage,
- OpCovOutputIdx($id, 0$) = 0, so the sole covenant output is at output index 0,

- OpAuthOutputCount$(i) = 1$ for the channel input,
- OpAuthOutputIdx$(i, 0) = 0$, so the channel input's sole authorised output is at output index 0,
- OpOutputCovenantId$(0) = id$ and OpOutputAuthorizingInput$(0) = i$, so output 0 directly declares continuation under the same covenant id and this input as its authoriser,
- successor output 0 has value V and OpTxOutputSpk(0) = P2SH(ContestScript$_m$); the witness exposes the successor's settlement slots (spk$_A, v_A$) and (spk$_B, v_B$), the covenant recomputes state_root$_m$ and settlement_mask$_m$ from those slots, rejects mask 0x00, and checks $v_A + v_B = V$,
- certificate (scope_id$, m,$ state_root$_m, \sigma_m$) validates,
- SEQ$_{\text{replace}}$ = le64(2^{63}) (disable bit set, every other bit zero).

Bounded shape:

- OpTxInputCount $\in \{1, 2\}$ and OpTxOutputCount $\in \{1, 2\}$.
- Inputs: [channel input at idx 0, optional sponsor input at idx 1].
- Outputs: [contest output of V at idx 0, optional sponsor change at idx 1].
- Sponsor invariant: sponsor_input = sponsor_change + fee.

Under the fixed-principal profile, the sponsor input is mandatory for any positive-fee transaction by algebraic consequence of the conservation rule $v_A + v_B = V$.

6.5 Settlement (unilateral, after relative-sequence maturity)

Spends ContestOutput(n). Produces Settlement(n) using the authenticated settlement-presence mask, so that no zero-valued participant output is emitted. Covenant predicates:

- OpCovInputCount$(id) = 1$,
- OpCovOutputCount$(id) = 0$,
- OpCovInputIdx$(id, 0) = i$ where i is the channel-input index, so the channel input is the unique covenant-authorising input,
- OpAuthOutputCount$(i) = 0$ for the channel input,
- the revealed ContestScript$_n$ authenticated by the spent input's P2SH commitment contains n, state_root$_n$, and settlement_mask$_n$,
- in the mask cases below, "output k pays spk" means OpTxOutputSpk(k) = toccataSPK(spk),
- if settlement_mask$_n$ = 0x03, output 0 pays spk$_A$ with value $v_A > 0$, output 1 pays spk$_B$ with value $v_B > 0$, and optional sponsor change is at output 2,
- if settlement_mask$_n$ = 0x02, output 0 pays spk$_A$ with value V, $v_B = 0$, and optional sponsor change is at output 1,
- if settlement_mask$_n$ = 0x01, output 0 pays spk$_B$ with value V, $v_A = 0$, and optional sponsor change is at output 1,
- settlement_mask$_n$ = 0x00 is rejected,
- the covenant recomputes state_root$'_n$ from the authenticated mask, the participant SPKs, and both participant balances, then asserts state_root$_n$ = state_root$'_n$. This binds the settlement's allocation to the state root committed by the spent contest output, so the on-chain predicate proves that each party receives the signed state's allocation rather than an arbitrary pair of values that sum to V,
- $v_A + v_B = V$ (a separate conservation predicate; the state-root recomputation above binds the allocation, but does not replace the principal-conservation check),
- SEQ$_{\text{settle}}$ = le64(Δ) (disable bit clear, bits 32–62 zero, Δ in the low 32 bits).

Bounded shape:

- OpTxInputCount $\in \{1, 2\}$.
- For mask 0x03, OpTxOutputCount $\in \{2, 3\}$; for masks 0x01 and 0x02, OpTxOutputCount $\in \{1, 2\}$.
- Inputs: [contest input at idx 0, optional sponsor input at idx 1].

- Outputs: canonical participant outputs selected by $\mathsf{settlement\_mask}_n$, followed by optional sponsor change.
- tx.lock_time = 0.
- If a sponsor input is present, the sponsor invariant sponsor_input = sponsor_change + fee is enforced; under the fixed-principal profile, any positive-fee settlement requires the sponsor input by the same algebraic consequence of $v_A + v_B = V$ that applies to the replacement branch.

The mandatory-sponsor rule is a deliberate fixed-principal choice, not a claim that all deployable channels must use an external fee payer. It prevents fees from silently eroding the certified participant allocation, but it introduces sponsor availability, fee-market, and single-party fee-payment dependency assumptions. A deployment that wants participants to pay fees from channel value should instead adopt a fee reserve or a principal-changing transition; both variants change the conservation predicate and belong in the committed policy hash, with principal-changing transitions treated only as a future-work sketch in §10.

6.6 Cooperative Close (orthogonal fast path)

Spends OpenOutput or ContestOutput(n). Both parties sign the same $(\mathsf{scope\_id}, I, S)$ triple where I is the exact input outpoint and S is the canonical hash of the participant-settlement outputs. Let

$$S = H_{\text{KurrentCoopCloseOutputs/v1}}\big(\mathsf{settlement\_mask} \,\|\, \text{commitSPK}(\text{spk}_A) \,\|\, \text{le64}(v_A) \,\|\, \text{commitSPK}(\text{spk}_B) \,\|\, \text{le64}(v_B)\big)$$

be the canonical hash of the participant-output set under its own domain tag, and let

$$m_{\text{coop}} = H_{\text{KurrentCoopClose/v1}}(\mathsf{scope\_id} \,\|\, I \,\|\, S)$$

as defined in §5.4. Covenant predicates:

- $\mathsf{coop\_close\_enabled} = 1$ in the authenticated policy, checked by the policy-hash binding rule of §5.5,
- OpCovInputCount(id) = 1,
- OpCovOutputCount(id) = 0,
- OpCovInputIdx(id, 0) = i where i is the channel-input index,
- OpAuthOutputCount(i) = 0 for the channel input,
- the participant outputs follow the same mask-index rule as unilateral settlement, and $\mathsf{settlement\_mask} = \mathsf{0x00}$ is rejected,
- the witness exposes the canonical participant-output hash S; the covenant recomputes S' from the actual participant settlement outputs of this transaction using the formula above and asserts $S = S'$, so the cooperative-close signature is bound to the canonical participant allocation. Optional sponsor change is governed separately by the sponsor invariant and is not part of S,
- the witness exposes σ_{coop}; the covenant asserts SchnorrVerify($P_{\text{agg}}, m_{\text{coop}}, \sigma_{\text{coop}}$) = 1 with m_{coop} as above and P_{agg} the channel's fixed aggregate verification key, so the cooperative close is fully authorised by both participants under the same scope and outpoint,
- $v_A + v_B = V$ (a separate conservation predicate; the Schnorr authorisation binds the chosen output set, and this check ensures that the chosen set preserves the channel principal),
- all protocol and sponsor inputs use le64(2^{63}) (disabled relative sequence),
- tx.lock_time = 0.

Bounded shape:

- OpTxInputCount $\in \{1, 2\}$.
- For mask $\mathsf{0x03}$, OpTxOutputCount $\in \{2, 3\}$; for masks $\mathsf{0x01}$ and $\mathsf{0x02}$, OpTxOutputCount $\in \{1, 2\}$.
- Inputs: [channel input at idx 0, optional sponsor input at idx 1].
- Outputs: canonical participant outputs selected by $\mathsf{settlement\_mask}$, followed by optional sponsor change.

Cooperative close is fully authorised by both parties, has no contest race, no sequence wait, and is disjoint from the unilateral-replacement safety argument of §7.

7 Security Claims

This section states the security claims of the normative bilateral channel as a research note. Each claim is followed by a justification paragraph that names the predicate, UTXO, certificate, conservation, or sequence facts that discharge it. The claims are scoped to the consensus predicate and do not constitute a full formal proof; the deployment-level finality-depth assumption is stated in Claim 4, the liveness assumption is stated in §8, and the remaining operational obligations are recorded in §12 and §13.

Claim 1 (State authentication and singleton lineage). In any validation view reached from a valid Kurrent genesis, there is at most one live Kurrent covenant UTXO for the channel lineage. Each live contest output authenticates its own state number, state root, settlement mask, and static channel parameters through the spent input's P2SH commitment. *Justification.* The funding genesis of §6 creates exactly one OpenOutput(V) and makes output 0 the only initial authorised Kurrent output. Opening and replacement consume exactly one covenant input and create exactly one authorised output at index 0; they also require OpOutputCovenantId(0) = id and OpOutputAuthorizingInput(0) = i. Settlement and cooperative close consume exactly one covenant input and create zero covenant outputs. By induction, the live Kurrent lineage is singleton until it terminates. For a contest output, the revealed $\mathsf{ContestScript}_n$ is authenticated by the spent input's P2SH script public key, so n, $\mathsf{state\_root}_n$, and $\mathsf{settlement\_mask}_n$ are constants of the spent UTXO's script, not free witness fields. The script recomputes $\mathsf{scope\_id}$ from the actual input covenant id and the authenticated static parameters.

Claim 2 (Transition soundness). Every accepted replacement of ContestOutput(n) creates a uniquely covenant-bound ContestOutput(m) with authenticated $m > n$, a valid state certificate, conserved principal, and a successor script that carries the authenticated successor state. *Justification.* The replacement branch requires one covenant input, one covenant output, output 0 as the sole authorised output, direct output binding to the same covenant id and the current channel input, and OpTxOutputSpk(0) = P2SH($\mathsf{ContestScript}_m$). The successor script contains m, $\mathsf{state\_root}_m$, and $\mathsf{settlement\_mask}_m$. The branch checks $m > n$, verifies σ_m over $(\mathsf{scope\_id}, m, \mathsf{state\_root}_m)$, recomputes the successor root and mask from the supplied participant allocation, rejects the invalid mask, and checks $v_A + v_B = V$.

Claim 3 (Pre-maturity exclusion). Settlement of ContestOutput(n) is invalid before the DAA-relative maturity interval expires, while a valid higher-state replacement is immediately eligible. *Justification.* The settlement branch requires $\mathrm{SEQ}_{\mathrm{settle}} = \mathrm{le64}(\Delta)$ with the disable bit clear, so consensus rejects settlement until the spent contest output reaches creation DAA score plus Δ in the current validation view. The opening and replacement branches use disabled relative sequence values and do not wait on that maturity rule. The replacement must still satisfy the authenticated-state, certificate, successor-script, conservation, and singleton-lineage predicates of Claim 2.

Claim 4 (Conditional stale-state safety and finality). If an honest party holds a valid certificate for $m > n$ and a replacement transaction satisfying Claim 2 becomes accepted before a stale settlement of ContestOutput(n) becomes accepted, then no descendant view that preserves that accepted replacement can settle the allocation committed by ContestOutput(n). Once that replacement reaches the deployment's Kaspa finality policy, the exclusion holds in every finality-respecting descendant view, except under violation of the underlying consensus-finality assumption. *Justification.* This is the actual stale-state theorem: the later state wins only if its replacement spend is accepted first. After that acceptance, ordinary UTXO uniqueness removes ContestOutput(n) from the accepted UTXO set, so a later settlement of the same input is a duplicate-input spend against a spent output. The finality policy is Kaspa-native: a deployment may express it as a DAA-score, blue-score, or selected-parent/finality-depth rule defined by the target network profile, not as a Bitcoin-style "k confirmations" rule. The protocol provides no state-number priority after maturity; if stale settlement is accepted first, the higher certificate alone cannot reverse that accepted spend.

Claim 5 (Settlement allocation and zero-output soundness). Every accepted unilateral settlement pays exactly the participant allocation authenticated by the spent contest output, without requiring zero-valued transaction outputs. *Justification.* The settlement branch takes n, $\mathsf{state\_root}_n$, and $\mathsf{settlement\_mask}_n$ from the P2SH-authenticated $\mathsf{ContestScript}_n$. It reconstructs the participant output set selected by the mask, rejects mask 0x00, rejects any mask inconsistent with (v_A, v_B), recomputes $\mathsf{state\_root}'_n$ from the mask, SPKs, and balances, and asserts equality with $\mathsf{state\_root}_n$. The conservation check $v_A + v_B = V$ is separate. Masks 0x01 and 0x02 encode all-to-one terminal states with only one participant output, avoiding the consensus-invalid zero-output case.

Claim 6 (Cooperative-close non-interference). Cooperative close is fully authorised by both parties and binds to the exact input outpoint. It lies outside the unilateral stale-state claim and cannot be replayed to close a later contest output. *Justification.* The cooperative-close message $m_{\text{coop}} = H_{\text{KurrentCoopClose/v1}}(\mathsf{scope\_id} \,\|\, I \,\|\, S)$ from §5.4 binds the input outpoint I as one of its three bound fields. The Schnorr check

$$\text{SchnorrVerify}(P_{\text{agg}},\ m_{\text{coop}},\ \sigma_{\text{coop}}) \ =\ 1$$

is a covenant predicate of the cooperative-close branch. A later contest output in the same channel lineage has a different outpoint $I' \neq I$, so the same signed m_{coop} fails the Schnorr check when the witness presents I' as the spent outpoint. The cooperative-close branch is outside the unilateral-replacement claim. When it closes a ContestOutput(n), it consumes that contest input by joint authorisation and produces settlement outputs directly; it does not create a successor contest output and cannot be replayed against a later outpoint.

8 Race and Monitoring Model

The protocol predicate is stated in DAA-space:

$$a_{\text{replace}} \ <\ a_n + \Delta$$

where a_{replace} is the DAA score at which the replacement spend of C_n becomes accepted, and a_n is the DAA score of C_n's creation.

The scores are interpreted inside a particular validation view. If a reorganisation removes C_n, the channel is evaluated against the surviving lineage rather than against the abandoned timer. If a reorganisation removes a replacement before that replacement reaches the deployment's finality policy, the safety statement is evaluated against the surviving UTXO set; the monitor should re-evaluate the channel against that view and, if needed, publish a replacement or settlement transaction within that view's response window. Frequent reorganisations therefore increase the liveness and fee budget beyond the steady-state inclusion assumption; deployments that anticipate non-trivial reorganisation depth should record that tolerance in the channel's policy_hash alongside Δ. Once a replacement reaches the chosen finality policy, Claim 4 applies. The response-window state machine is reorg-aware under accepted-view UTXO semantics: timers and spends are facts of the current accepted view until finality fixes them under the deployment assumption.

After relative-sequence maturity, both the settlement transaction and a higher-state replacement may be individually valid conflicting spends of the same contest UTXO. The covenant cannot guarantee that the higher state wins merely because its state number is larger; whichever conflicting spend consensus accepts first consumes the UTXO. The protocol therefore provides no state-number priority after maturity. A higher-state replacement must be accepted before the stale settlement is accepted, full stop. The equal-DAA tie-breaker question does not arise as a channel-level rule.

Wall-clock latency is a deployment-level operational assumption:

$$\Pr\big[T_{\text{detect}} + T_{\text{construct}} + T_{\text{propagate}} + T_{\text{include}} < T_{\Delta}\big] \ \geq\ 1 - \varepsilon$$

Here T_{Δ} is the expected wall-clock duration of the DAA-score interval Δ at the target network profile, with stated confidence. As a deployment-level example, the post-Crescendo Kaspa target of 10 BPS (one block every 0.1 s) gives $T_{\Delta} \approx 0.1\,s \cdot \Delta$. A conservative $\Delta = 600$ DAA scores therefore corresponds to roughly a 60 s replacement window; a tighter $\Delta = 60$ corresponds to roughly 6 s. The protocol does not pick a Δ here; the deployment chooses it, and the operational confidence ε is a deployment-level artefact, not a consensus-level parameter.

Publication is insufficient: the replacement must not merely be signed and broadcast. Mempool acceptance is insufficient: a transaction in the mempool is not yet a spent UTXO. Watchtower observation is insufficient: an observed transaction may be censored, replaced, or evicted. The replacement must become an accepted spend before the stale settlement does. Censorship, reorganisation, fee-market dynamics, and watchtower economics are deployment-level concerns that the consensus predicate does not solve cryptographically; the protocol separates the consensus predicate $a_{\text{replace}} < a_n + \Delta$ from the deployment-level probabilistic assumption $\Pr[\cdot] \geq 1 - \varepsilon$ and treats them as distinct obligations.

9 Factories as Compression of Many Channels

This section is research-only. It records the invariants a future compressed factory must satisfy; it is not an implemented production factory and it is not part of the bilateral channel's normative safety claim.

A channel factory [2] generalises the channel idea. Instead of placing one bilateral channel directly on-chain, several virtual channels are represented inside a shared covenant state. The base layer sees a compact factory state; the participants see many internal relationships. The attraction is obvious: one on-chain object can support many off-chain relationships.

The risk is equally obvious. Compression must not blur ownership. If one virtual channel is materialised out of the factory, the untouched channels must remain exactly what they were. A factory therefore adds a second invariant on top of latest-state replacement:

$$\text{factory principal} = \sum_{\text{active virtual channels}} \text{virtual principal}.$$

This is not merely an off-chain accounting equation. Under the chosen factory representation, the equality must be cryptographically checkable. A Merkle-sum style commitment, an aggregate commitment, a proof-carrying materialisation path, or an equivalent committed internal representation can serve that role; a plain membership root does not prove principal conservation unless sums are also committed or otherwise verified.

Materialisation is sound only if it preserves that equality. The touched channel exits; its value is paid or moved according to the materialisation rule; every untouched channel remains byte-for-byte the same; the factory's remaining principal is reduced by exactly the touched principal. The conservation rule is less elegant than the state-number rule but no less central. Without conservation, a factory is not compression; it is ambiguity.

Factory non-interference is not guaranteed by the conservation equality alone. The materialisation path must carry enough committed state and proof material to show that the touched virtual channel exits under its rule, the remaining factory principal is reduced by exactly the touched principal, no new virtual claims are injected, and every untouched virtual channel remains identical under the committed representation. These are implementation audit targets, not decorative accounting rules.

This note focuses on materialisation safety: how a touched virtual channel exits without corrupting untouched virtual channels. Kurrent's paper-level factory claim is limited to materialisation invariants and proof obligations. A complete factory design must also specify the update protocol by which virtual channel updates alter the committed factory state. That includes the internal representation, which parties authorise each update, and how the next aggregate or factory commitment is produced. If the factory uses a compressed representation, non-interference must be proved through committed state and proof material, not asserted through accounting equations.

Scope of the present evidence. The current local-devnet factory evidence uses a full-state typed model in which every virtual channel and the total principal are recomputed by the verifier against a complete materialisation plan. This is a bounded local-devnet evidence path for materialisation invariants: it demonstrates the structure of the model-level guarantees the compressed factory has to discharge explicitly, but it is not yet a production compressed Merkle-sum or aggregate-commitment factory. A production deployment must replace the full-state recomputation with a Merkle-sum style or equivalent committed internal representation, so that materialisation carries a proof against the factory's public commitment rather than depending on the verifier having the full pre-state. That is a separate design slice and is out of scope for the present research note.

The factory principle is therefore a natural companion to covenant identifiers. The covenant lineage says that the factory remains the same public object. The materialisation rule says what may be removed from it. The preservation rule says what must not be touched. Together they let a complex off-chain object present a simple on-chain boundary.

10 Future Work

The normative construction in §5–§6 is intentionally narrow. The following items are required for a complete production protocol but are out of the present normative scope; they are listed here so that the boundary between the spec and the production gate is visible.

Hashlock interop. Both legs share the same payment hash $h = H(r)$, and redemption on either leg reveals the same preimage r. Each local claim binds that shared hash to its own amount, expiry, direction, and keys. This note does not specify Lightning route-hop, PTLC, or adaptor-signature integration; those are separate protocol surfaces that build on top of the same witness-scoping discipline.

Key rotation and quorum changes. The fixed-participant bilateral channel of §5–§6 is the production mainline. Rotating the participant key set, supporting m-of-n thresholds with $n > 2$, or any participant-set transition would

require an EpochTransitionCertificate or equivalent, with its own commitment hierarchy. The extension is in scope for a future protocol specification and is not part of the present channel mainline.

Factories and sum commitments. A channel factory compresses multiple bilateral relationships behind one on-chain object. Factory materialisation safety is the principal conservation

$$\text{factory principal} = \sum_{\text{active virtual channels}} \text{virtual principal}$$

discharged by a Merkle-sum style commitment, an aggregate commitment, a proof-carrying materialisation path, or an equivalent. Factories are out of the bilateral core; the design considerations are recorded in §9 and the design note `KURRENT_FACTORY_COMMITMENT_DESIGN.md`.

KIP-21 observability and proof systems. The post-Toccata partitioned sequencing surface is valuable for watchtower evidence, application-local proving, and future based-app and zk proof paths. It is not a fund-safety primitive for the bilateral channel, but a production deployment that needs monitoring or compressed evidence will benefit from integrating it as an observability layer. The contest-output transaction graph and the marker / KIP-21 accepted-ordering surface play complementary roles. The contest UTXO, the authenticated state carrier, the singleton-lineage predicates, and the relative-sequence maturity rule discharge the bilateral fund-safety argument of §7; no marker, lane, or accepted-ordering proof is required for that conditional stale-state theorem. The marker / KIP-21 path is the substrate for the prototype harness of §13 and the optional evidence layer for the based-app, compressed-factory, and zk-proof compositions above. A lane-local accepted-ordering commitment is useful where the protocol needs compact event history, portable proof objects, or watchtower observations, rather than merely a valid conflicting spend.

Normative contest-graph implementation. The contest-output transaction graph specified in §6 is the production mainline; the local-devnet harness exercises a marker-and-verifier path as prototype evidence. A production-grade implementation of the normative contest graph, with full script bytecode, runtime cost model, and consensus-fee economics, is the next milestone beyond the present research note.

Principal-changing and factory extension surfaces (sketch). The bilateral core of §5–§6 fixes the channel principal V and the fixed-participant set; the following two extensions are forward hooks only and are not normative predicates. *Principal-changing transitions.* A splice-in or splice-out would be a new authorised transition type, replacing ContestOutput(n, V) with ContestOutput(m, V') under the same scope_id and a valid aggregate authorisation. The successor state_root would commit to participant slots summing to V', and the covenant would check that the successor contest output also has value V'. The stale-state exclusion argument of §7 is unchanged; only the conservation predicate changes. *Factory transitions.* A compressed factory would wrap the bilateral contest-output channel as a materialisable virtual leaf, consuming a FactoryOutput(root, total), proving membership and principal of a touched virtual channel, and creating a successor FactoryOutput(root′, total $- V$) whose untouched leaves are preserved under a Merkle-sum or equivalent commitment. The bilateral channel safety claim of §7 applies only after materialisation; the factory safety claim is separate and must additionally prove sum conservation and untouched-leaf non-interference. Both extensions are sketched here, not specified; their normative treatment belongs to a future `KURRENT_FACTORY_COMMITMENT_DESIGN.md` or equivalent slice, and the production kernel remains the fixed-principal, fixed-participant bilateral channel.

Multi-participant generalisation. The contest-output kernel is also neutral to the participant count. The same construction generalises from a bilateral 2-of-2 channel to a fixed k-participant k-of-k channel by replacing the two participant slots with a canonical participant vector, replacing $v_A + v_B = V$ with $\sum_{i=0}^{k-1} v_i = V$, and deriving P_{agg} from the ordered participant public-key list. The stale-state argument of §7 is structurally unchanged: a higher-numbered valid replacement must satisfy the authenticated-state, singleton-lineage, conservation, and response-window predicates, and must be accepted before stale settlement. Threshold t-of-k authorisation, participant-set changes, and key rotation require an explicit authorisation-policy layer or EpochTransitionCertificate and are treated as separate extensions.

Forward reference: compressed factory verification path. When the factory evidence path eventually moves from full-state model verification to a compressed commitment, the natural on-chain verification layer for the materialisation transition is a KIP-16 wrapper [6]. The Merkle-sum or equivalent commitment records the factory's internal asset structure; a KIP-16 proof is designed to turn the model-level factory findings (overclaimable

touched leaves, injectable after-state channels, unenforced principal sum, untouched-leaf interference) into explicit proof obligations rather than into per-leaf recomputation; and the covenant script is designed to bind the proof's public inputs to the actual spent factory output, the successor output, the current factory identity, the covenant lineage, and the settlement output hash, so that a valid proof for a different factory or a different transition cannot be replayed. The combination is a bounded on-chain proof verification path: KIP-16 verifies the computation, the covenant script binds the computation to the transaction, and the Merkle-sum root is the structure the computation is over. This paragraph is a forward reference, not a current claim. It is not part of the production-readiness gate for the normative bilateral channel; it is not part of the current marker evidence mainline; and the carrier for the factory root, the choice of proof system, the KIP-16 cost and pricing model, the third-party dependency model, and the security assumptions are all left to a future `KURRENT_FACTORY_COMMITMENT_DESIGN.md` milestone.

11 Eligibility, Not Retaliation

The non-confiscatory claim is best stated as an eligibility rule. The covenant does not classify the publisher's behaviour. It classifies the candidate: same scope_id, authenticated contest state, strictly higher state number for replacement, valid certificate, conserved value, valid settlement mask, and response-window survival. If those checks fail, the candidate is not entitled to final settlement. This removes confiscatory revocation from the protocol logic, but it does not remove deterrence or liveness pressure entirely: the stale publisher can no longer enforce the stale settlement if a higher authorised spend is accepted in time, and honest parties still need monitoring capacity to make that happen. Claims 1–6 of §7 together discharge that classification: Claim 1 authenticates the state carrier and singleton lineage, Claim 2 gives transition soundness, Claim 3 gives pre-maturity exclusion, Claim 4 states the conditional stale-state theorem and finality corollary, Claim 5 binds settlement allocation without zero-valued outputs, and Claim 6 handles the cooperative-close fast path.

This matters because a Kaspa-shaped construction has to make the replacement decision public without importing Bitcoin's floating transaction mechanism. The candidate that wins is not the one with the most sympathetic history, the largest sponsor fee, or the most convenient indexer trace. It is the one that satisfies the named eligibility rule and is accepted first by consensus. The shift is in the consequence: the response path presents the later valid entitlement, rather than a revocation secret that confiscates the stale publisher's funds.

The design still needs active monitoring. A participant, delegate, or watchtower must surface the higher authorised state before the response window closes. The response window is the liveness interval in which the higher state must become an accepted spend; the liveness proposition of §7 is a deployment-level operational assumption about how likely the interval is to be satisfied under realistic propagation, censorship, and reorganisation conditions.

12 Limitations and Open Questions

The bilateral safety core specified in §5–§6 is now stated as a consensus-level predicate: the P2SH-authenticated state carrier, the KIP-20 singleton-lineage base case, the covenant successor predicate, the relative-sequence maturity rule, the MuSig2 aggregate authorisation, the settlement mask, and the cardinality checks together give the conditional stale-state safety statement of Claim 4. The fund-safety primitive does not require a consensus-recognised accepted-ordering surface.

What is not yet a complete production system is the operational layer: liveness under realistic propagation, censorship, fee-market, and reorganisation conditions, plus the wider protocol surface (factories, key rotation, hashlock interop, monitoring substrate). Production readiness for the bilateral channel additionally requires deployment-level parameterisation of the response window and finality depth; the production fee-market and sponsor policy; watchtower or monitoring architecture with a stated liveness budget; and operational key management. The post-Toccata partitioned sequencing surface is one of several valid monitoring substrates; it is not a fund-safety gate for the bilateral channel.

Named protocol-specification requirements (not research-note gaps). The following items are requirements that a production protocol specification must satisfy; they are not gaps in the present research note, and are recorded here so that the boundary between the two documents is visible. *(i)* The response-window state machine must specify the half-open interval $[a, d)$ semantics, the DAA-score interpretation of maturity, and the validation-view semantics at the boundary. The protocol provides no state-number priority after maturity; conflicting valid spends are resolved by ordinary consensus acceptance. *(ii)* Safety must be conditional on a bounded-inclusion and fee-market assumption: an explicit fee reserve or externally attachable sponsor path; an anchor/child or package-publication fee-bumping mechanism; a congestion model; a censorship/inclusion assumption; and a watchtower fee budget. Without such an assumption, no finite response window is provably safe. *(iii)* Same-number conflict

handling must specify what "first" means under consensus finality — first consensus-finalised certificate wins, safe fallback to the prior state, or explicit channel failure — rather than treating arrival order at the verifier as authoritative. *(iv)* Operational key management must specify the signing-ceremony procedure, the recovery path, and the rule under which the fixed-participant set is rotated; the present mainline fixes a single fixed-aggregate key for the channel lifetime. *(v)* The factory design must add fixed-width arithmetic with explicit overflow rejection, range-constrained balances, canonical leaf ordering with unique virtual-channel identifiers, and a Merkle-sum or equivalent principal-commitment that the materialisation proof can verify against. *(vi)* The hashlock interop must define the shared-secret scoping rule, the per-leg claim commitment, and the witness-binding discipline; the present note fixes only the shared-secret claim form (§10). *(vii)* The deployment parameter set (response-window length Δ, finality-depth policy, sponsor-policy id, coop_close_enabled flag) must be specified for the target network and recorded in the channel's policy_hash.

The conceptual claim is narrower and stronger: the bilateral safety primitive is the contest-output state machine of §6. The remaining items above are deployment obligations and adjacent protocol surfaces, not bilateral-safety gaps. Eltoo supplies the replacement philosophy. Kurrent combines these into a Kaspa-native settlement model: state is negotiated locally, while settlement eligibility is granted only to the state that survives a public, monotone, covenant-bound rule.

13 Production-Readiness Scope

Kurrent as a research artefact contains three distinct protocol surfaces that share a covenant lineage but have different production-readiness dependencies. The normative bilateral contest-output channel specified in §5–§6 is the production mainline and has the smallest primitive footprint. The existing local-devnet harness exercises a marker-and-verifier prototype evidence path that demonstrates model-level displacement behaviour (§7) but is not the normative construction. The factory surface is a separate, more demanding protocol surface (§9).

Protocol surface	**Accepted-ordering surface required?**
Normative bilateral contest-output channel	No
Prototype marker evidence path	Yes (the harness depends on it)
Future compressed factory / based-app / zk	Possibly / depends on the composition

Table 1: *Accepted-ordering surface requirement by protocol surface. The normative bilateral channel of §6 does not require a consensus-recognised accepted-ordering surface for fund safety: the contest-UTXO exclusivity plus the covenant cardinality predicates plus the relative-sequence maturity rule are sufficient. The prototype marker evidence path requires an accepted-ordering substrate because the harness records and verifies candidates through the marker flow. A future compressed factory, based-app, or zk-proof composition may need a sequencing commitment for compressed evidence and global reconstructibility, depending on the proof system chosen.*

Accepted-ordering surface: who needs it? The normative bilateral channel of §6 and the prototype marker evidence path are different artefacts. Conflating them would force the normative channel to depend on a prototype substrate, or would force the prototype to claim production-grade fund safety. Kurrent keeps them separate. The post-Toccata partitioned sequencing surface [9] is one valid substrate for the prototype and the future-factory / based-app surfaces; the normative channel does not depend on it.

Production-readiness dependencies (normative bilateral channel).

1. Deployment parameter set: response-window length Δ in DAA-score units, finality-depth policy, sponsor-policy id, coop_close_enabled flag.
2. Production fee-market and sponsor policy, including the rule that the lower stale settlement may pay a larger sponsor fee and still lose to the higher-state replacement accepted first by consensus.
3. Watchtower or monitoring architecture, with a stated liveness budget and the cross-participant response path.
4. Operational key management: signing ceremony, recovery path, fixed-participant set rule.
5. Independent review of the covenant bytecode, the consensus predicate, and the script-engine cost model.

Production-readiness dependencies (prototype marker evidence path). The harness exercises lane binding, candidate evidence reconstruction, sponsor-accounting checks, and model-level displacement behaviour. It does not implement the normative contest-output transaction graph; it is the prototype evidence path. Its production-quality upgrade is a normative-graph implementation (§10), at which point the harness becomes a regression suite rather than the mainline.

Production-readiness dependencies (future compressed factory / based-app / zk). The factory surface requires a principal-commitment (Merkle-sum or equivalent) and a materialisation proof path. A based-app composition may use the post-Toccata partitioned sequencing surface for application-local proving with reconstructible global ordering, but the normative bilateral channel does not. A zk-proof composition inherits whatever accepted-ordering commitment its proof system requires; the matrix above is the source-of-truth split between the channel mainline and the proof substrate.

14 Conclusion

The elegance of latest-state settlement is that it treats the ledger as a judge of present entitlement, not as a historian of every intermediate agreement. Kaspa's covenant trajectory makes this question worth taking seriously. Transaction introspection supplies visibility. Covenant identifiers supply lineage. DAA-relative sequence maturity supplies the response window. Kurrent combines these into a Kaspa-shaped language: a fixed-participant bilateral channel whose fund safety rests on a contest-output state machine with a fixed aggregate verification key. The principle can be put plainly. A channel is safe when yesterday's valid agreement cannot defeat today's valid agreement, and the contest-output state machine is the primitive that makes that sentence public.

References

[1] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. *eltoo: A Simple Layer2 Protocol for Bitcoin*. Blockstream and Lightning Labs, 2018. https://blockstream.com/eltoo.pdf.

[2] Conrad Burchert, Christian Decker, and Roger Wattenhofer. *Scalable Funding of Bitcoin Micropayment Channel Networks*. Royal Society Open Science, 2018. https://doi.org/10.1098/rsos.180089.

[3] Jonas Nick, Tim Ruffing, and Yannick Seurin. *MuSig2: Simple Two-Round Schnorr Multi-Signatures*. CRYPTO 2021; IACR ePrint 2020/1261. https://eprint.iacr.org/2020/1261.

[4] Maxim Biryukov and Ori Newman. *KIP-10: New Transaction Opcodes for Enhanced Script Functionality*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0010.md.

[5] Michael Sutton. *KIP-14: The Crescendo Hardfork*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0014.md.

[6] Alexander Safstrom. *KIP-16: New Transaction Opcodes for Verifiable Computation (OpZkPrecompile)*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0016.md.

[7] Ori Newman. *KIP-17: Covenants and Improved Scripting Capabilities*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0017.md.

[8] Michael Sutton. *KIP-20: Covenant IDs*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0020.md.

[9] Michael Sutton, Maxim Biryukov, and Hans Moog. *KIP-21: Partitioned Sequencing Commitment with O(activity) Proving*. Kaspa Improvement Proposals, vendored at commit 1aba3b8. https://github.com/kaspanet/kips/blob/1aba3b8/kip-0021.md.

[10] Kaspa. *Build on Kaspa: Toccata: covenants & zk*. Accessed 2026-06-20. https://kaspa.org/build.

[11] Michael Sutton. *Kaspa Covenants++ "Toccata" Hard-Fork Outlook*. Medium, April 3, 2026; accessed 2026-06-20. This post precedes the pinned KIP snapshot used in this note; the canonical KIP text is the vendored `kaspanet/kips@1aba3b8` snapshot. https://medium.com/@michaelsuttonil/kaspa-covenants-toccata-hard-fork-outlook-a4d81a40900c.

[12] Stefan Dziembowski, Lisa Eckey, Sebastian Faust, Julia Hesse, and Kristina Hostáková. *Multi-Party Virtual State Channels*. EUROCRYPT 2019; IACR ePrint 2019/571. https://eprint.iacr.org/2019/571.